Research Document

Research Question:

How do CPG frequency, gait, coupling strength, terrain and oscillator affect the locomotion performance and stability of a quadrupedal robot?

Hypothesis:

1. Frequency will not fall over at medium value and will significantly affect performance.
2. The gait will impact efficiency and will moderately affect performance
3. The gradient will slightly affect performance
4. Coupling strength will cause the robot to fall at low values and it will significantly affect performance
5. The oscillator will significantly affect performance
6. There are strong interactions between these parameters

Independent Variables

- Frequency
- Gait (Phase - Offset)
- Coupling
- Terrain gradient
- Oscillator
- Terrain Type

Measured DVs:
- Time (t)
- Distance (d)
- Robot mass (m)
- Voltage ~~and current~~ at intervals ($V(t)$)
- Current at intervals ($I(t)$)
- Joint positions ($\dot{q}$)
- Joint Velocity ($\ddot{q}$)
- IMU orientation (Θ)
- CoM position ($[x, y, z]$)
- Foot position (P)

Calculated DVs:
- Speed $= \left(\frac{d}{t}\right)$
- Power average $= \frac{\int_0^t (V(t)\, I(t))\, dt}{t}$
- Energy per metre $= (\text{Energy}/d)$
- Cost of trasport $= \left(\frac{E}{m \times g \times d}\right)$
- Energy $= \left(\int_0^t (V(t)\, I(t))\, dt = E\right)$
- RMS tilt $= \left(\sqrt{\frac{\Sigma(\theta_i - \bar{\theta})^2}{N}}\right)$
- CoM oscillation $(y) = \left(\sqrt{\frac{\Sigma(y_i - \bar{y})^2}{N}}\right)$

PTO →

- Amplitude $(\phi) = \max(\phi) - \min(\phi)$
- ZMP Dev $= \sqrt{(x_{ZMP} - x_{ref})^2 + (y_{ZMP} - y_{ref})^2}$
- Stride length $= \max(P_z) - \min(P_z)$, assuming local frame
- Duty cycle $= \frac{T_{stance}}{T_{stride}}$
- Foot slip ≈ 0
- Phase error $(\Delta\phi) = \phi_{actual} - \phi_{desired}$
- Tracking error $= \sqrt{\frac{\Sigma(q_{actual} - q_{desired})}{N}}$

Experiment Overview:

I will first ~~run~~ run a 2162 trial mujoco simulation and collect the data. This data will be passed through a ~~pthy~~ python statistical analyser from which conclusions can be drawn.

I will then build a 16 DOF quadruped and run the same code on the physical robot but only run the main/important tests to limit time (2162 trials would take a long time). The ruun a statistical analysis to look for connections. The compare sim to real life.

__/__/____

Measured DVs:

- Time (t)
- Distance (d)
- Robot mass (m)
- Voltage ~~and current~~ at intervals ($V(t)$)
- Current at intervals ($I(t)$)
- Joint positions ($\vec{q}$)
- Joint Velocity ($\dot{q}$)
- IMU orientation (Θ)
- CoM position ($[x, y, z]$)
- Foot position (p)

Calculated DVs:

- Speed $= (\frac{d}{t})$
- Power average $= \frac{\int_0^t (V(t) I(t))\, dt}{t}$
- Energy per metre $= (\text{Energy} / d)$
- Cost of transport $= \left(\frac{E}{m \times g \times d}\right)$
- Energy $= (\int_0^t (V(t) I(t))\, dt) = E$)
- RMS tilt $= \left(\sqrt{\frac{\sum (\theta_i - \bar{\theta})^2}{N}}\right)$
- CoM oscillation (y) $= \left(\sqrt{\frac{\sum (y_i - \bar{y})^2}{N}}\right)$

PTO →

__/__/____

- Amplitude (ϕ) = max (ϕ) − min (ϕ)
- ZMP Dev $= \sqrt{(x_{ZMP} - x_{ref})^2 + (y_{ZMP} - y_{ref})^2}$
- Stride length = max (p_z) − min (p_z), assuming local frame
- Duty cycle $= \frac{T_{stance}}{T_{stride}}$
- Foot slip ≈ 0
- Phase error ($\Delta\phi$) = $\phi_{actual} - \phi_{desired}$
- Tracking error $= \sqrt{\frac{\sum (q_{actual} - q_{desired})}{N}}$

Experiment Overview:

I will first ~~run~~ run a 2162 trial mujoco simulation and collect the data. This data will be passed through a ~~pkg~~ python statistical analyser from which conclusions can be drawn.

I will then build a 16 DOF quadruped and run the same code on the physical robot but only run the main/important tests to limit time (2162 trials would take a long time). The run a statistical analysis to look for connections. The compare sim to real life.

Theoretical foundation

A CPG is an involuntary form of biological, rhythmic locomotion, in this context walking.

An oscillator is a dynamical system that produces a repeating pattern.

A simple oscillator:
$x(t) = A \sin(\omega t + \phi)$

where
$x(t)$ = output
A = amplitude
ω = angular frequency
t = time
ϕ = phase offset

Although this is an oscillator, it not very useful for a CPG gait as it cannot converge to a natural position after perturbations

A dynamical system is a system that describe how something changes over time.

For example
$\frac{dx}{dt} = f(x)$, the rate of change of x depends on x

This can be done for 2 variables

$\frac{dx}{dt} = f(x, y) = \dot{x}$, $\frac{dy}{dt} = g(x, y) = \dot{y}$

Both equations show that the future state depends on the current state.

x and y are state variables where a state is everything you need to determine what happens next.

A static system states: At $t = 3.2$, oscillator should be here.
A dynamic system continously evolves: $(x, y) \rightarrow (x + \Delta x, y + \Delta y)$

To understand oscillators, they are not plotted x vs t but rather y vs x. A closed orbit in a graph is

called a limit cycle.

- Limit cycle
A limit cycle is the closed trajectory that a system approaches. If the gait is distrubed, it will return to its original orbital.

Hence, limit cycles are useful for locomotion - it solves the problem of static systems.

- Hopf oscillator:
We can start of with the Hopf oscillator

The equation is:
$\dot{x} = (\mu - r^2)x - \omega y$
$\dot{y} = (\mu - r^2)y + \omega x$
where $r^2 = x^2 + y^2$

x is the first oscillator state
y is the second oscillator state
(x, y) can be used to describe oscillator position

$\dot{x} \equiv$ rates of change of x
$\dot{y} \equiv$ rate of change of y

- Angular Frequency
ω is the angular frequency.
This controls the speed of the limit cycle
Using frequency, $\omega = 2\pi f$.
This could be used as an IV

- What is μ
μ controls the radial behaviour of oscillators with $\mu - r^2$ determins whether the oscillators moves towards or away from the radius.

- Proof of $\mu - r^2$
Assume $\mu = 1$, $r = \sqrt{\mu} = \sqrt{1} = 1$.
If $r > 1$, $\mu - r^2 > 0$ so the system pushes outwards
If $r < 1$, $\mu - r^2 < 0$ so the system pushes inwards
If $r = 1$, $\mu - r^2 = 0$ so the system is stable
Let $r^* \equiv$ desired radius so $r^* = \sqrt{\mu}$

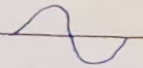
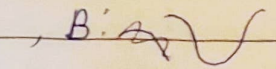
- Amplitude
Amplitude acts as a scaling factor for the oscillator
$q(t) = q_0 + A\,x(t)$

where:
$q(t) \equiv$ des. measured joint angle at time t

q_0 ≡ neutral state of the joint
A ≡ scaling factor (amplitude)
$x(t)$ ≡ oscillator output

- Phase:

Phase is the shift from A to B and is represented by ϕ.

A:  , B: 

The phase is measured via angles where a full cycle is 2π.

Assume:
$\phi_A = 0$, $\phi_B = \pi$

This means B is $\frac{\pi}{2\pi} = \frac{1}{2}$ a cycle behind A.

Through these phase offsets, the gait (walk, pace, trot) can be controlled.

- Coupling:

To generate quadrupedal movement, multiple oscillator are required. The way the oscillators

Coupling strength is represented by w_{ij}

where:
i ≡ recieving oscillator
j ≡ influencing oscillator

If $w_{ij} = 0$, i and j do not influence eachother
If $w_{ij} = 0$, i and j influence eachother greatly.

- Kuramoto Model:

The phase coupling (Kuramoto model) can be shown as:

$$\dot{\phi}_i = \omega_i + \sum_j w_{ij} \sin(\phi_j - \phi_i)$$

where:
$\dot{\phi}_i$ ≡ rate of change of phase of oscillator i
ω_i ≡ natural frequency of i
w_{ij} ≡ coupling strength from j to i
$\phi_j - \phi_i$ ≡ phase difference
$\sin(\phi_j - \phi_i)$ ≡ direction and magnitude of phase correction

The Kuramoto model dictates the interaction between oscillators. It does not have an oscillator itself, only the interactions.

$e^{i\pi} + 1 = 0$

Project -1: make an animation of a Hopf oscillator
Project -2: ~~Apply~~ 2 oscillators to a leg
Project -3: Introduce phase relations for a quadruped

Task 1: Basic Hopf
Task 2: two uncoupled Hopf
Task 3: 2 Hopf with diffusive coupling
Task 4: 2 Hopf with phase shifted coupling
Task 5: Kuramoto model

Task 1:

This was quite easy as all I needed to do is implement the equation. The only hurdle was ~~B~~ putting the oscillator on a graph which I did with matplotlib

Task 1.5:

I went on a bit of a tangent where I tried to add inertia to the point. The solution to do this is quite simple, use the equation to calculate a target velocity, use $\alpha(\Delta v/m)$ to control acceleration and the multiply by dt to get the actual velocity and use that to get the position.

Task 2:

This is similar to task 1 however some structural changes were made. Since there were 2 points, I duplicated the required variables and I made a Hopf function that is independent of the pointer.

Task 3:

Diffuse coupling is a form of coupling where the oscillators are 'attracted' to eachother by an extra force being applied ($k(x_2 - x_1)$) where x_2 is the x-position of the other point and x_1 is the current position of the point. I added this to the velocity calculations. To make sure that new point positions were accurate, I ~~made~~ saved the previous positions of the points and used those points as a reference for the oscillator. I forgot to add that I am using Euler integration for the positions instead of RK4 because Euler is simpler ($dy = \frac{dy}{dx} \cdot dx$) which can be shown as ($x_1 = v_{x_1} \cdot dt$). This form of coupling showed that the follower point will eventually reach the master point. But I also found that the radius of the limit cycle does not change.

Task 4:

For my original Phase shifted coupling, I used a rotation matrix and multiplied it by the ~~position~~ state vector of the other point. I seperated the axis for the program but the theory remains the same. I took this as the coupling value and used diffuse coupling to add it to the velocity. I also graphed the phase difference ($\phi_2 - \phi_1$) across to to see how it converges to a value. However, changing the k values causes a change in the limit cycle radius

~~Task 4.5~~

Kuramoto Model (refresh):

The kuramoto model is a framework that dictates how oscillators syncronise and interact with eachother. The equation that governs these interactions:

$$\frac{d\theta_i}{dt} = \omega_i + \frac{K_{ii}}{N} \sum_{j=1}^{N} \sin(\theta_j - \theta_i + \alpha)$$

ω_i

$\omega_i \equiv$ angular frequency (same as Hopf oscillator)
$K_{ij} \equiv$ the weight of coupling between oscillators i
$N \equiv$ number of connected oscillators
$\theta_a \equiv$ angle of oscillator a
$\alpha \equiv$ phase difference target

Task 5:

As shown before the tasks, the equation is

$$\dot{\phi}_i = \omega_i + K_{ij} \sin(\theta_j - \theta_i + \alpha)$$

To implement this, I took the code from Task 4 and ~~replaced~~ removed the diffusive coupling and replaced ω_i of the velocity calculation with $\dot{\phi}_i$. This solved the T4 problem of a changing radius with different coupling. It also allows me to see the change in phase difference which is now, much faster

=

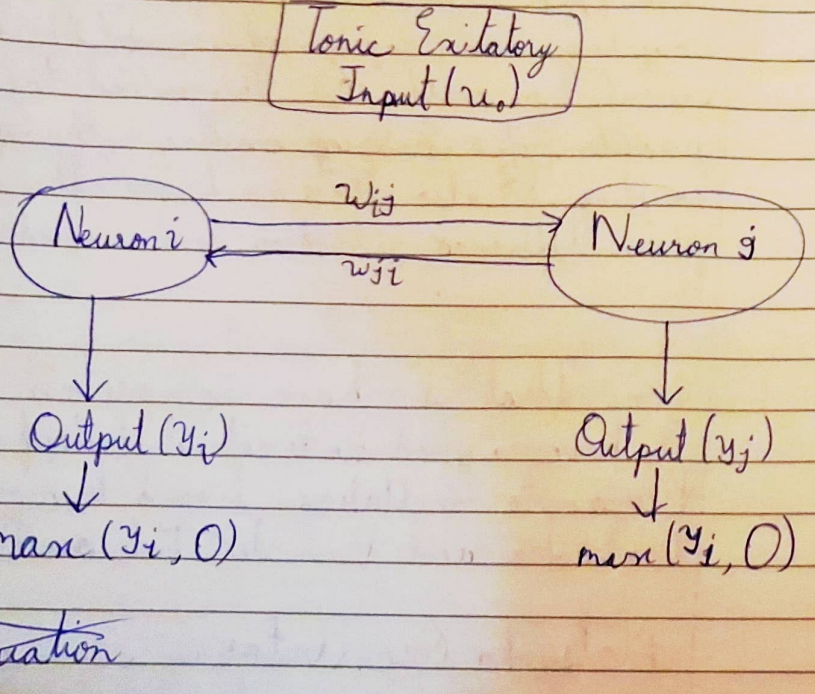
Now that I have completed the tasks, I have a good understand of Hopf and Kuramoto oscillators. I need to move onto Matsuoka and Van der Pol oscillators.

Matsuoka Oscillator:

The matsuoka oscillator consists of at least 2 neurons. Both neurons recieve a tonic exitory input (u_0).

The 'synaptic' connections have negative weights so when neuron i is active, it supresses neuron i (w_{ij}). As neuron i is active, it fatigues causing its output to decrease until it cannot supress neuron j (v_j). Then, only the positive value is taken

Dyer B Diagram:



Equation

In the oscillator, neuron i can be thought of as a flexor neuron and neuron j can be thought of as an extensor neuron.

- Equations:

Since 2 neurons are used, 2 sets of ODE describes the oscillators.

Neuron 1 (Membrane and fatigue):

$$\tau_x \dot{x}_1 = -x_1 - by_1 - a_{12} z_2 + s_1$$
$$\tau_y \dot{y}_1 = -y_1 + z_1$$
$$z_1 = \max(x_1, 0)$$

Neuron 2 (Membrane and fatigue):

$$\tau_x \dot{x}_2 = -x_2 - by_2 - a_{21} z_1 + s_2$$
$$\tau_y \dot{y}_2 = -y_2 + z_2$$
$$z_2 = \max(x_2, 0)$$

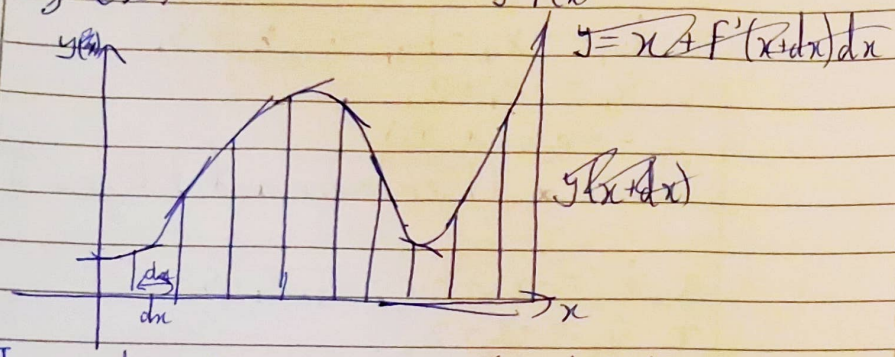
The ($z_i = \max(x_i, 0)$) statment makes sure that the output is positive

= Variable meanings:

$x_i \equiv$ Internal state of neuron i
$y_i \equiv$ Fatigue value of neuron i
$z_i \equiv$ Non-negative output signal (Firing rate)
$s_i \equiv$ Constant excitatory stimulus
$a_{ij} \equiv$ Synapic inhibitory weight from j to i
$b \equiv$ Scaling parameter of fatigue
$\tau_x, \tau_y \equiv$ Scale of x and y velocity response

Euler integration:

In euler integration, the simulation runs similar to regular integration. the timespan is split into dt. The ~~integral~~ derivative is calculated at $t+dt$ and then added to the y value after being multiplied by dx.



The equation is shown as: $y = x + f'(x+dx)\,dx$

Since x_1, y_1, x_2 and y_2 change over time, we can assume them as functions and use function notation. Hence $a_i(t)$ means the 'a' value of neuron i at time t.

By rearranging the formula, we get

$$\dot{x}_1 = \frac{-x_1 - b y_1 - a_{12} z_2 + s_1}{\tau_x}$$

and

$$\dot{y}_1 = \frac{-y_1 + z_1}{\tau_y}$$

By using Euler integration equation, we get:

$$x_1(t+\Delta t) = x_1(t) + \dot{x}_1 \Delta t$$

$$= x_1(t) + \left(\frac{-x_1(t) - b y_1(t) - a_{12} z_2(t) + s_1}{\tau_x}\right)\Delta t$$

$$y_1(t+\Delta t) = y_1(t) + \dot{y}_1 \Delta t$$

$$= y_1(t) + \left(\frac{-y_1(t) + z_1(t)}{\tau_y}\right)\Delta t$$

and the same can be done for neuron 2

$$x_2(t+\Delta t) = x_2(t) + \left(\frac{-x_2(t) - b y_2(t) - a_{21} z_1(t) + s_2}{\tau_x}\right)\Delta t$$

$$y_2(t+\Delta t) = y_2(t) + \left(\frac{-y_2(t) + z_2(t)}{\tau_y}\right)\Delta t$$

These equations also have requirements to converge to a stable limit cycle

1: Timescale Separation Threshold

The adaptation of y must react significantly slower than x so that the neurons don't fire simultaneously and cause it to fall into a static spiral. The mathematical requirement is:

$$\tau_y > 3\tau_x$$

2: Destabilising Mutual Inhibition

The inhibitory weights must be strong enough to destabilising nodes, preventing neurons from firing simultaneously and constatly. The flexor and extensor neurons cancel eachother out, locking up the joint. The mathmatical constraint is

$$a > 1 + \frac{\tau_x}{\tau_y}$$

3: Input Boundedness Zone

To prevent one neuron from permanently overpowring the other neuron, the ratio of $s_2 : s_1$ must be

$$\frac{a}{1+b} < \frac{s_2}{s_1} < \frac{1+b}{a}$$

Implementation of Matsuoka Oscillator:

To graph the oscillators, I set up 2 uncoupled Oscillators to work in parallel and assigned each oscillator an axis. Assuming the constraints are met, the Oscillators reached a stable orbit.

~~Now, the final oscillator I need to look into is~~

The next step is to control the phase of each oscillation which can be done through a Kuramoto model. The main problem with using this method is that The kuramoto model is phase only, so it consists of 1D while Matsuoka is a state space model and has 4D. This dimensional mismatch makes the

integration of the two systems. This further cascades to cause another mismatch: the Kuramoto model expects sinusoids while Matsuoka oscillations are not sinusoidal.

Also, due to Matsuoka's constraints, many of the variables have to be tuned correctly and Kuramoto model must not push these variables out of the bifurcation zone.

Also, Finally, Matsuoka doesn't have a phase so we need to make our own definition.

Fix:
To make a 4D state space into 1D, an infinitesimal phase response curve (iPRC) must be derived, its just a tool to describe the phase of a self sustaining oscillator. To calculate the iPRC, you first write the differential equation for the ith pair to find the stable limit trajectory $x_i(t)$ with the natural frequency ω_i. The iPRC is solved by solving the adjoint equation for Matsuoka oscillator.

These next 2 steps are slightly confusing

Next, we define the coupling function

When two neurons are coupled, they were able to interact together physically. The amount the j affects i is shown as P_{ij}. Using this variable, we can create an equation.

$$\Gamma_{ij}(\theta_i - \theta_j) = Z(\theta_i) \cdot P_{ij}(\theta_i, \theta_j)$$

The raw push (p_{ij}) depends on the phase difference and the strength j is pulling on i.

The sensitivity ($Z(\theta_i)$) determines the effect the raw push will have, on neuron i and i's timing.

The result (Γ_{ij}) is the effective change of phase, calculated from the dot product of $Z(\theta_i)$ and P_{ij}.

Now that 4D has been coverted into 1D, the Matsuoka oscillator can be added to a to the Kuramoto model

There also exists a second way of Hybridising the systems: The numerical method.

Notation $u_{i,1}$ means u of neuron 1 of oscillator i

$$\tau_1 \frac{du_{i,1}}{dt} = -u_{i,1} - a y_{i,2} - b f_{i,1} + s_0 + C_i(t)$$

$$\tau_a \frac{df_{i,1}}{dt} = -f_{i,1} + y_{i,1}$$

and so on...

Outputs are: $y_n = \max(0, u_{i,n})$

$C_i(t)$ the external coupling unit of the flexor neuron to control timing

$$C_i(t) = K \cdot R(t) \cdot \sin(\Psi(t) - \theta_i(t) + \phi_{ij})$$

where
ϕ_i = phase of neuron i
$\Psi(t)$ = averge phase of all legs
ϕ_{ij} = The gait offset
$R(t)$ = This shows how synchronised the oscillators are
K = Coupling strength.

How to calculate each value

$$\theta_i(t) = \arctan\left(\frac{u_{2,i}}{u_{1,i}}\right)$$

To calculate the global metrics, all of the phases should be coverted into complex vectors via $e^{i\theta} = \cos\theta + i\sin\theta$. All of the complex vectors are averaged. From this averge (Z):

$$Z = \frac{1}{N} \sum_{j=1}^{N} (\cos(\theta_j) + i\sin(\theta_j))$$

$$R(t) = |Z|$$

$$\Psi(t) = \arg(Z)$$

For ϕ_{ij}, there must exist a master leg to which all of other legs must follow or they reference the av $\Psi(t)$. Often, ϕ_{ij} is ~~as~~ hardcoded in an $N \times N$ matrix where N is the number of osillations and ϕ_{ij}'s value is located at (i,j) in the matrix

~~Trot~~:

To $\phi_{trot} = \begin{bmatrix} 0 & \pi & \pi & 0 \\ \pi & 0 & 0 & \pi \\ \pi & 0 & 0 & \pi \\ 0 & \pi & \pi & 0 \end{bmatrix}$